

OverWatch

Agentless, Read-Only, Self-Hosted Cloud-Native Application Protection Platform

System Design, Architecture & Implementation Guide

Platform version: v2.35.0

Document version: 1.0

Date: 31 July 2026

Status: Draft for review

Classification: Internal

Document Control

Field	Value
Title	OverWatch — System Design, Architecture & Implementation Guide
Product	OverWatch CNAPP (repo: AWS-Security-Scanner)
Platform version	v2.35.0 (aws_live_scanner.VERSION)
Test posture	2169 backend tests (mock boto3, no credentials) · 181 frontend vitest
Audience	Platform / security engineers, architects, operators
Prepared with	Grounded in the repository source at time of writing

How to use this document

This guide is organized top-down: the design charter and the invariants it enforces come first, because every downstream decision in OverWatch follows from them. The architecture chapters then describe the scanning engine, data model, governance layer, hosted control plane, and interfaces; the final chapters cover security, deployment, implementation, and testing. Appendices provide reference tables (modules, environment variables, API routes, IAM, graph model).

Table of Contents

Open in Microsoft Word and choose References → Update Table (or right-click the field below → Update Field) to build the page-numbered contents.

Document Control.....	2
How to use this document.....	2
Table of Contents.....	2
Executive Summary.....	6
1. Introduction & Scope	6
1.1 Purpose	6
1.2 What OverWatch Is (and Is Not)	6
1.3 Positioning & Differentiators	7
1.4 Reference Environment	7
2. Design Charter & Principles	7
2.1 The Four Pillars.....	7
2.2 Zero-Telemetry & Air-Gap	8

2.3 The Frozen Correlator & Display-Only Overlays	8
2.4 Fail-Closed Everywhere	9
2.5 Pure, Dependency-Injected, Offline-Testable	9
3. System Architecture Overview	9
3.1 Logical Tiers.....	9
3.2 Runtime Topology	10
3.3 Technology Stack	11
4. Core Scanning Engine.....	12
4.1 The Scan Pipeline	12
4.2 The Security Graph.....	12
4.3 Internet Exposure & L7 Reachability	14
4.4 Deep-Plane Ingestion (Buy, Don't Rebuild)	14
4.5 Attack-Path Correlation & Choke Points.....	14
4.6 Effective-Permissions Ceiling	15
4.7 CIEM & Least-Privilege	15
4.8 Agentless Workload Side-Scan (CWPP)	15
4.9 Kubernetes Posture (KSPM / KIEM)	15
4.10 Data Security Posture (DSPM) & AWS-Resident Secrets	15
4.11 AI Security Posture (AI-SPM).....	16
4.12 The Runtime Ingest Planes (CDR / EDR / Malware).....	16
4.13 Cloud-Forensics Timeline	16
4.14 Grounded-RAG Copilot	16
5. Data Model & Persistence	17
5.1 One Backend, Two Dialects.....	17
5.2 What Is Persisted	17
5.3 Display-Only Configuration (No Schema Churn).....	17
6. Query & Governance Layer	17
6.1 WQL — the Security-Graph Query Language	17
6.2 Controls & Policy-as-Code.....	18
6.3 SAMPLE == LIVE Parity	18
7. Supply Chain & Container Registry	18

7.1 SBOM Ingest, Diff, License & VEX	18
7.2 Agentless ECR Registry Scanning	18
7.3 Non-AWS OCI Registry Connectors	18
8. Hosted Control Plane	19
9. External Interfaces (HTTP API)	19
10. Frontend Architecture	20
11. Security Architecture	20
11.1 Egress Containment & Zero-Telemetry	20
11.2 Secret Handling	21
11.3 SSRF & Redirect Guards	21
11.4 IAM Trust Model	21
11.5 Fail-Closed Authentication	21
12. Deployment & Operations	22
12.1 Topology	22
12.2 The Shipped Deploy Path	22
12.3 Configuration (Environment Variables)	22
12.4 Air-Gapped Install	22
12.5 Amazon Linux Notes	23
12.6 Shift-Left: CI/CD Gate & IDE	23
13. Implementation Guide	23
13.1 Repository Layout	23
13.2 The Core Pattern: Pure Module + Injected Seam	23
13.3 Adding a Posture Check	23
13.4 Adding an Ingest Normalizer or Overlay	24
13.5 The SAMPLE == LIVE Rule	24
13.6 Coding Conventions	24
14. Testing & Quality Assurance	24
14.1 The Suite	24
14.2 The Guards	24
14.3 The Rigor Loop	25
15. Roadmap & Extensibility	25

Appendix A — Module Inventory	26
Appendix B — Graph Model Reference.....	27
Traversable attack edges (E_PATH)	27
Annotations (enrich a node; never walked as a hop)	27
Appendix C — Selected API Routes	27
Appendix D — Glossary.....	28

Executive Summary

OverWatch is a Cloud-Native Application Protection Platform (CNAPP) for AWS that is deliberately agentless, read-only, and self-hosted. It audits a customer's AWS estate over the AWS APIs alone — no agent, no eBPF sensor, no active exploitation, no data-plane reads — and turns thousands of flat findings into a ranked handful of end-to-end attack paths to the resources that actually matter (the “crown jewels”).

Its differentiator is the security graph and the attack-path correlation engine built on it: rather than scoring findings in isolation, OverWatch scores the entry→target chain, so a high-CVSS vulnerability on an unexposed host with no path to sensitive data never surfaces as a false critical. Around that spine it layers effective-permissions analysis (the true IAM ceiling after permission boundaries and SCPs), agentless workload vulnerability scanning (CWPP), Kubernetes posture (KSPM/KIEM), data security posture (DSPM), AI security posture (AI-SPM), and “ingest” planes that fold a customer's existing runtime, malware, and vulnerability signals onto the same graph without OverWatch ever running a sensor of its own.

OverWatch is engineered to run in the most constrained environments a security team operates: it is zero-telemetry and air-gap installable, ships fully offline, and confines every network primitive to a small, test-enforced allowlist. The platform is delivered as a hardened, non-root container that runs on a private-subnet EC2 instance inside the customer's security VPC and reaches AWS exclusively through VPC endpoints.

Bottom line. OverWatch trades the breadth of an agent for sovereignty and signal quality: a self-hosted, read-only platform that a regulated or air-gapped organization can run entirely within its own boundary, and that answers “what can actually be reached, and what would it take to reach the crown jewels?” rather than emitting an undifferentiated finding firehose.

1. Introduction & Scope

1.1 Purpose

This document is the authoritative design and architecture reference for OverWatch. It describes the system's guiding charter, its logical and runtime architecture, each major engine, the data and persistence model, the query and governance layers, the hosted control plane, the external interfaces, and the security, deployment, implementation, and testing models. It is intended to let an engineer reason about, operate, extend, and safely deploy OverWatch without reading the entire source tree first.

1.2 What OverWatch Is (and Is Not)

OverWatch IS	OverWatch is NOT
Agentless — audits via AWS APIs only	An agent / eBPF runtime sensor

Read-only — never mutates a scanned resource	A remediation actuator that changes cloud state
Self-hosted — runs inside the customer boundary	A multi-tenant SaaS that ingests customer data
AWS-first CNAPP with an attack-path graph	A flat compliance checklist scanner
Zero-telemetry / air-gap installable	A phone-home / cloud-callback product
A platform that ingests existing runtime signals	A DAST / active-exploitation tool

1.3 Positioning & Differentiators

- **Attack-path first.** Ranking is on the entry→crown-jewel chain, not the finding — this is what collapses a finding firehose into a ranked few.
- **Effective permissions, not attached policies.** The true IAM ceiling is computed as identity policies intersected with permission boundaries and Service Control Policies.
- **Agentless CWPP.** OS-package and language-dependency vulnerabilities are matched against a BYO OSV feed and folded onto the same graph, lighting up attack paths even when Amazon Inspector is disabled.
- **In-charter substitutes for what an agent would give.** Rather than build a runtime sensor (which would break the charter), OverWatch ingests the customer's existing CrowdStrike / Falco / GuardDuty-Runtime / malware output and correlates it onto the graph.
- **Sovereign by construction.** Zero-telemetry, air-gap installable, and a single, test-enforced network-egress allowlist.

1.4 Reference Environment

Unless stated otherwise, this document describes platform version v2.35.0. The engine runs on Python 3.10+; the shipped container image is built on python:3.12-slim. The state store runs on SQLite (single-node) or PostgreSQL (HA) behind one dual-dialect backend abstraction. The web console is a React single-page application served as static assets.

2. Design Charter & Principles

The charter is the set of load-bearing invariants that define OverWatch. They are not aspirational; they are enforced in code and in the test suite, and every architectural decision is downstream of them. A change that would violate the charter is treated as a defect, not a trade-off.

2.1 The Four Pillars

Pillar	Meaning	Why it is load-bearing
Agentless	Audit via AWS control-plane APIs only; no agent, no eBPF, no data-plane read	Preserves the sovereign wedge and keeps the blast radius of OverWatch itself near zero

Read-only	Never mutate a scanned resource; no active exploitation / DAST	A security tool must never be the cause of an incident in the estate it audits
Self-hosted	Runs entirely inside the customer boundary	No customer data leaves the boundary; suits regulated / air-gapped orgs
AWS-first	AWS is the primary, deeply-modeled cloud	Depth over breadth — the attack-path graph is AWS-native

2.2 Zero-Telemetry & Air-Gap

OverWatch performs no telemetry, no phone-home, and no build-time or run-time package fetch. Every network primitive in the codebase is confined to a small allowlist enforced by a recursive-AST tripwire test (`tests/test_zero_telemetry.py`) that runs over every shipped module. Only three files may hold a network primitive, and each has a single, documented purpose:

Allowlisted egress file	Sole purpose
<code>aws_kube.py</code>	Read-only Kubernetes API reads for KSPM/KIEM (operator-configured seam)
<code>cnapp_connectors.py</code>	Outbound notifications to the operator's OWN tools (Jira/Slack/...)
<code>aws_layer_fetch.py</code>	Registry layer-blob pulls — ECR presigned S3 + non-AWS OCI registries

Everything else is either an AWS SDK (boto3) call to the customer's own account or a local file read. Offline packaging is a pinned wheelhouse plus a `--no-index` container build; the complete egress inventory lives in `NETWORK.md` and the sealed-install procedure in `docs/AIRGAP_RUNBOOK.md`.

2.3 The Frozen Correlator & Display-Only Overlays

The attack-path correlation engine (`aws_correlate.py`) is treated as byte-frozen: its traversable-edge set and crown-jewel logic are pinned by a checksum test so that no feature can silently change how risk is scored. New capabilities that surface signal — saved-query Controls, policy-as-code, EDR/DSPM coverage, non-AWS registry findings — are added as display-only overlays that are appended to the finding catalog at read time with WARN status. They never enter the posture score (which counts FAIL only and is baked at scan time) and never enter the traversable graph.

Invariant. A display-only overlay can inform an operator but can never change the posture score or the attack-path ranking. This is what lets the platform grow quickly without destabilizing its core risk signal.

2.4 Fail-Closed Everywhere

- **Auth.** The hosted API ships fail-closed — every route returns 403 until a real `current_principal` (IdP/JWT) and a secret resolver are injected.
- **Secrets.** A credential is always a `secretsmanager://` or `ssm://` reference; plaintext is resolved transiently at point of use, never persisted, never returned over the API.
- **Partial data.** A partial or corrupt image rootfs, a truncated layer, or a missing package database fails closed (the image is dropped) rather than scanning to a false-clean.
- **Integrity.** A tampered or unsigned vulnerability feed is rejected fail-closed when a public key is supplied.

2.5 Pure, Dependency-Injected, Offline-Testable

Engine and control-plane modules are written as pure functions over injected seams: the AWS session, the HTTP poster, the secret reader/writer, and the clock are all passed in. This is why the entire 2169-test backend suite runs with mock boto3 and never touches a socket or an AWS credential, and why the same modules can be exercised deterministically against hand-built graphs and fixtures.

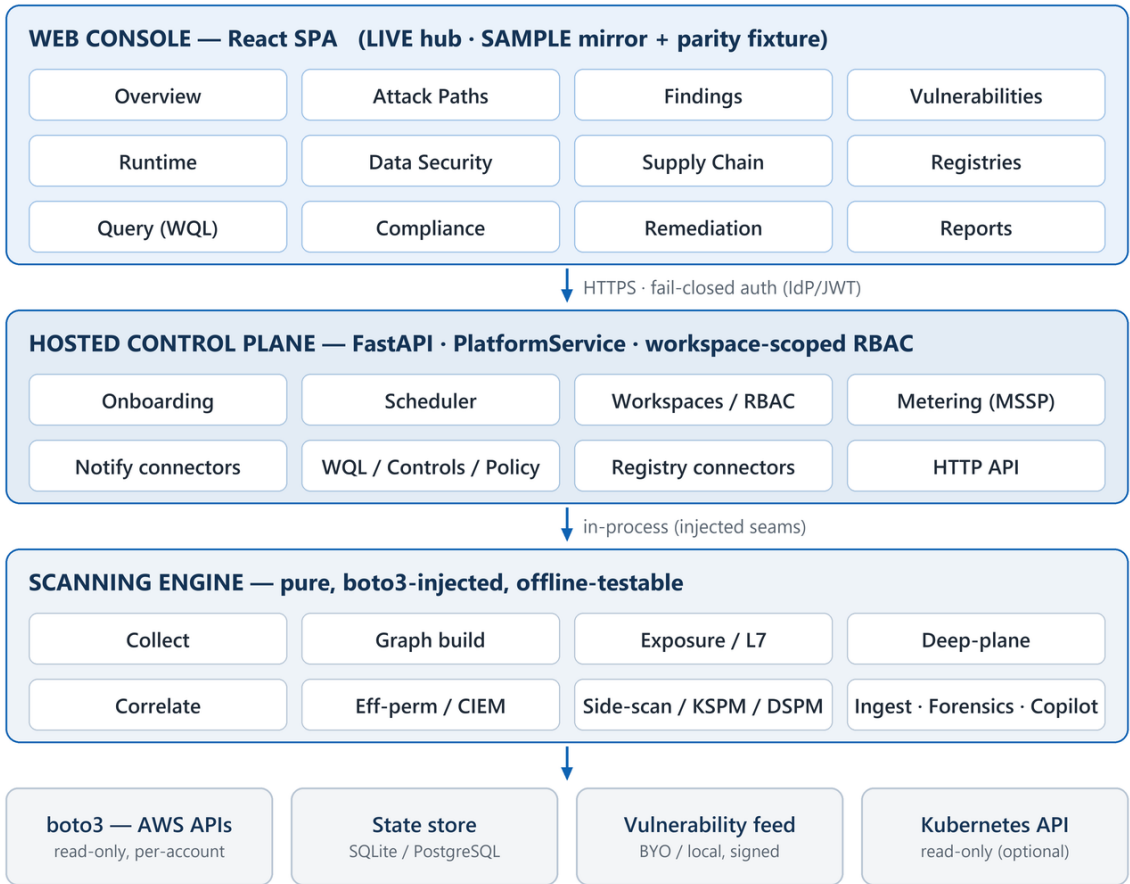
3. System Architecture Overview

3.1 Logical Tiers

OverWatch is a three-tier system: a scanning engine, a hosted control plane, and a web console. The engine is a library of pure modules that produce a scan result (findings, the security graph, attack paths, posture). The control plane wraps the engine with onboarding, scheduling, persistence, multi-tenancy, RBAC, metering, and an HTTP API. The console is a React SPA that renders the control plane's API and, in demo mode, an identical client-side mirror of the engine's matchers.

Figure 1 — Logical Architecture (three tiers)

A React console over a fail-closed control plane over a pure scanning engine



Charter: agentless · read-only · self-hosted · AWS-first · zero-telemetry / air-gap

Figure 1 — Logical architecture: a React console over a fail-closed control plane over a pure scanning engine, which reaches AWS (and optional seams) read-only.

3.2 Runtime Topology

In production the control plane and engine run together as one container on a private-subnet EC2 instance in the customer's security (tooling) VPC — the “hub.” The hub assumes a read-only role in each audited “spoke” account via sts:AssumeRole and reaches every AWS service through VPC interface/gateway endpoints, so no public-internet egress is required. The console is exposed only inside the VPC (typically behind an internal load balancer that terminates TLS and enforces the customer's identity provider).

Figure 2 — Runtime Topology (hub in the security VPC)

One hardened container on a private subnet; AWS via VPC endpoints; spokes via AssumeRole

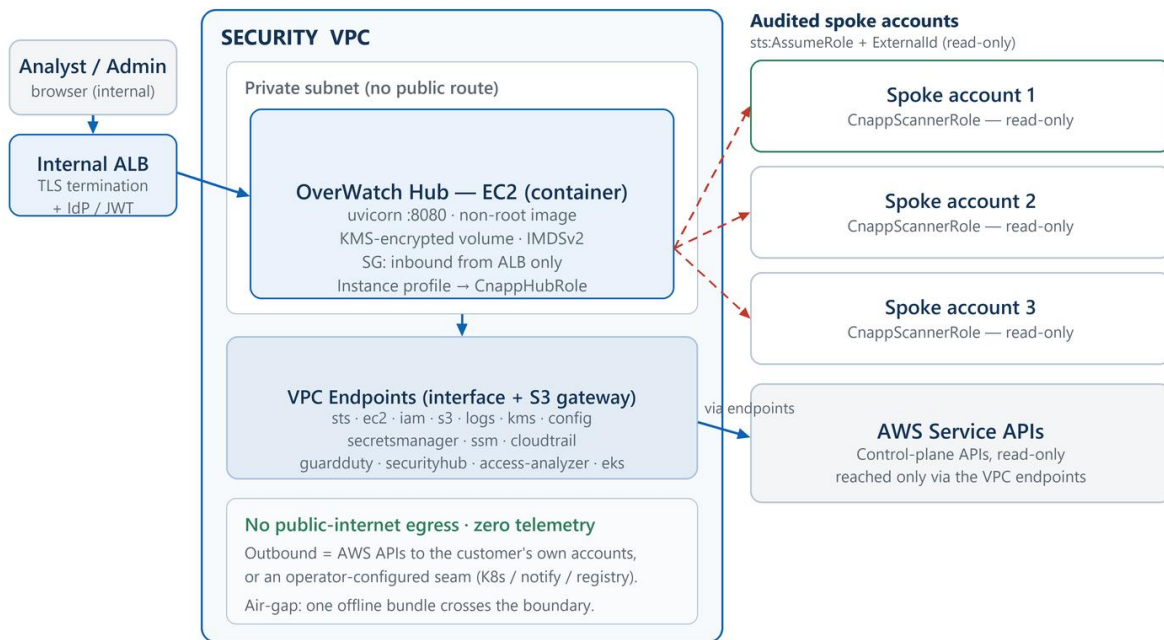


Figure 2 — Runtime topology: one hardened container on a private subnet reaches AWS via VPC endpoints and assumes read-only roles into each spoke account; no public-internet egress.

3.3 Technology Stack

Layer	Technology
Engine language	Python 3.10+ (container: python:3.12-slim)
AWS access	boto3 / botocore (read-only)
API framework	FastAPI + Starlette on uvicorn (pure-Python h11 build)
Data validation	pydantic v2
State store	SQLite (stdlib) or PostgreSQL via psycopg3 — one Backend abstraction
Web console	React single-page application (Vite build, static assets)
Packaging	Multi-stage, non-root Docker image; offline wheelhouse; --no-index build

4. Core Scanning Engine

The engine is a pipeline of pure modules. A scan collects read-only facts from AWS, builds a security graph, layers exposure/reachability, ingests the deep plane, and correlates the result into ranked attack paths and choke points. The subsections below describe each stage and the specialized posture engines that hang off the graph.

4.1 The Scan Pipeline

- **Collect.** Read-only enumeration of the estate across 44 resource sections via boto3, degrading gracefully where a permission is absent.
- **Graph build.** Resources become typed nodes; identity, exposure, vulnerability, data, and threat relationships become typed edges (`aws_graph.py`).
- **Exposure & L7 reachability.** A four-gate oracle decides what is genuinely reachable from the internet, and at layer 7, before anything is called “exposed” (`aws_exposure.py`).
- **Deep-plane ingestion.** Findings from Inspector, Macie, GuardDuty, and Access Analyzer are folded in as graph signal rather than re-implemented (`aws_deepplane.py`).
- **Correlate.** The graph is collapsed into ranked entry→crown-jewel attack paths and choke points (`aws_correlate.py`).
- **Persist & report.** Findings, posture, the graph, and attack paths are written to the state store; drift and waivers are reconciled; reports and the ATT&CK-style views are rendered.

Figure 3 — Scan Pipeline & Engine Fan-out

Read-only collect → graph → reachability → deep-plane → correlate; engines read the graph

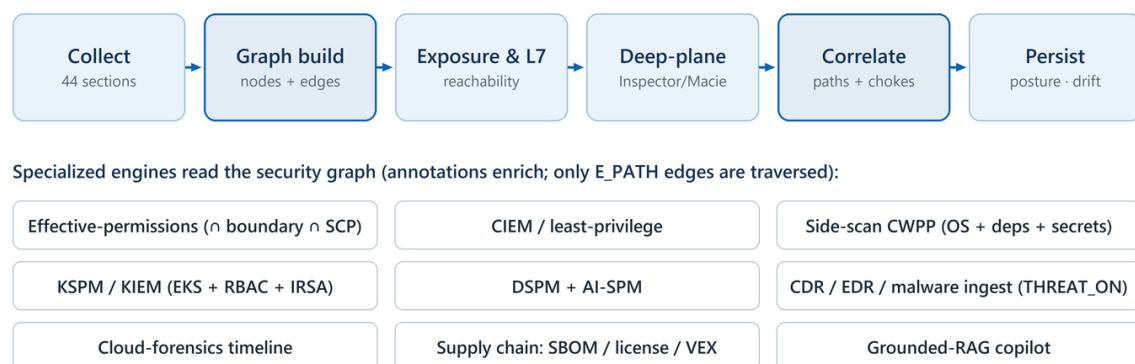


Figure 3 — The scan pipeline and engine fan-out: read-only stages build the graph; the specialized posture engines all read from it (annotations enrich; only E_PATH edges are traversed).

4.2 The Security Graph

The security graph is the backbone data structure. Every capability in OverWatch either adds nodes/edges to it or reads from it. Node kinds span identities, compute, network fronts, data

stores, Kubernetes objects, vulnerabilities, and threats; edges encode both traversable “attack” relationships and non-traversable annotations.

Category	Representative node kinds
Identity	IAMRole, InstanceProfile, ServicePrincipal, FederatedPrincipal, AnyPrincipal, AdminCapability
Compute / workload	LambdaFunction, ECSFargateTask, ECSTaskDefinition, NetworkInterface
Network front	LoadBalancer, CloudFrontDistribution, ApiGateway, InternetSource, ObservedCidr, DanglingDNSRecord
Data & secrets	S3 / DataStore kinds, Secret, SecretsManagerSecret, KMSKey
Containers / K8s	ECRRepository, ECRImage, KubePod, KubeServiceAccount, KubeAdminCapability
Risk overlays	Vulnerability / cve, ThreatFinding, CdrDetection

Edges are split into a traversable attack set and everything-else annotations. Only the traversable set is walked when computing attack paths; annotations (a vulnerability on a node, a threat detection, a technology tag) enrich a node without creating a lateral-movement hop.

Traversable edge set (E_PATH). EXPOSED_TO · ATTACHED_TO · HAS_INSTANCE_PROFILE · HAS_ROLE · CAN_ASSUME · CAN_PRIVESC_TO · CAN_READ_DATA · TARGETS. Annotations such as HAS_VULN, THREAT_ON, RUNS_IMAGE, and HAS_TECH enrich nodes but are never walked as hops.

Figure 4 — The Attack-Path Graph (the differentiator)

Rank the entry→crown-jewel chain; a toxic combination is a conjunction — a missing factor collapses the score

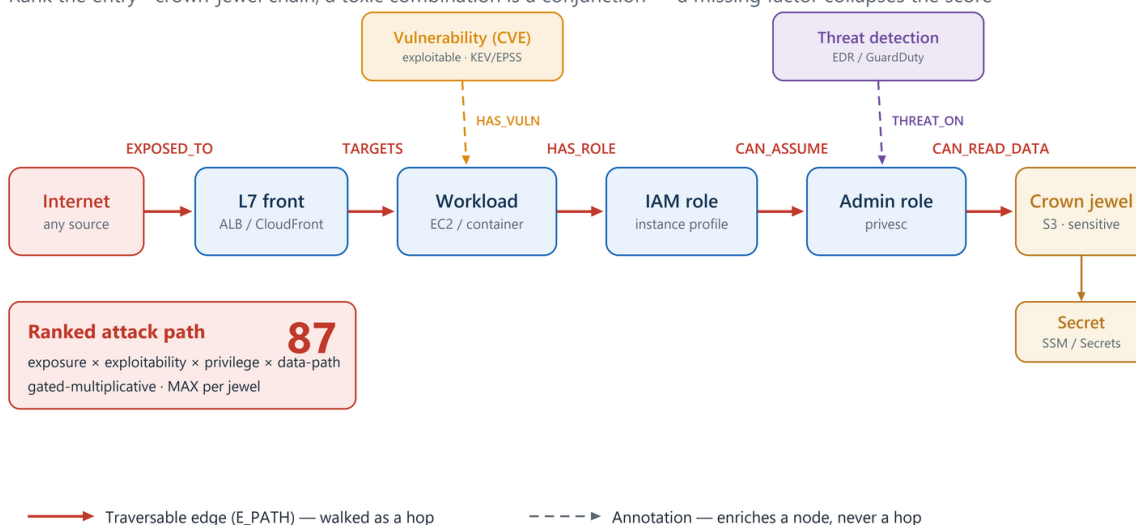


Figure 4 — An end-to-end attack path over traversable (solid) edges; annotations (dashed) enrich nodes. The score is gated-multiplicative, so a missing factor collapses the path — see §4.5.

4.3 Internet Exposure & L7 Reachability

Before OverWatch calls anything “exposed,” a four-gate reachability oracle must agree: a network path must exist (security group + NACL + route), the resource must present a listener, a layer-7 front (ALB/CloudFront/API Gateway) must actually route to it where applicable, and no compensating control must sever it. This low-false-positive posture is what lets EXPOSED_TO be a trusted attack-path entry edge rather than a noisy guess. A static security-group micro-segmentation lens (config-only, zero new grant) and an optional observed-traffic overlay from VPC Flow Logs complement the oracle without being folded into the traversable set.

4.4 Deep-Plane Ingestion (Buy, Don't Rebuild)

OverWatch does not re-implement AWS's own detectors. Where the customer already runs Amazon Inspector (vulnerabilities), Macie (data classification), GuardDuty (threats), or IAM Access Analyzer (external access), those findings are ingested and attached to the graph as native signal — an Inspector CVE becomes a HAS_VULN edge that can light up an attack path; a Macie sensitivity score contributes to crown-jewel classification. This keeps OverWatch additive to the customer's existing AWS-native investments.

4.5 Attack-Path Correlation & Choke Points

Correlation is the “ship the product” layer. It reads the graph and collapses it into the ranked handful of end-to-end paths that matter, then identifies choke points — single nodes whose remediation severs many paths to many crown jewels. Its methodology is deliberate and grounded in a verified research pass:

- **Score the path, not the finding.** The unit of ranking is an entry→target chain.
- **Gated-multiplicative scoring.** A toxic combination is a conjunction of exposure AND exploitability AND privilege/reach AND a path to crown-jewel data; factors multiply, so any missing dimension collapses the score — killing the classic “high-CVSS but unexposed, no data path” false positive.
- **MAX-per-jewel aggregation, never SUM.** Summing shared hops inflates benign infrastructure to fake-critical — the most common documented failure mode.
- **Explainable.** Every 0–100 score decomposes into its hop factors and driving findings; a path with no renderable rationale is a bug.
- **Choke points by dominance.** Ranked by severity-weighted path frequency, with a true-choke flag when every path to a target passes through the node (a graph dominator).

The engine is pure and boto3-free: the graph predicates (is_unconditioned, is_exploitable, node_has_threat) are injected as callables, so correlation is unit-tested against hand-built graphs and can never diverge from the emitters it ranks.

4.6 Effective-Permissions Ceiling

A principal's real power is not what its attached policies say — it is the intersection of its identity policies with any permission boundary and the account's Service Control Policies. OverWatch computes this effective ceiling (`aws_effperm.py`) and uses it to prune the graph: a `CAN_ASSUME` or `CAN_PRIVILEGE_TO` edge that the boundary or an SCP actually denies is removed, so attack paths reflect what is genuinely possible, not what a policy document superficially grants.

4.7 CIEM & Least-Privilege

On top of the effective-permissions solver, the CIEM slice (`aws_leastpriv.py`) right-sizes identities: it correlates granted permissions against access-analyzer last-accessed data to surface unused access and generates a least-privilege policy proposal. This is analysis and recommendation only — consistent with read-only, OverWatch never applies a policy change.

4.8 Agentless Workload Side-Scan (CWPP)

The side-scan (`aws_sidescan.py`) is OverWatch's answer to workload vulnerability coverage without an agent. It inventories a workload's OS packages and language dependencies, matches them against a vulnerability feed with ecosystem-correct version comparison (`dpkg` / `rpm` / `apk` and language ecosystems), and detects on-disk secrets — producing `HAS_VULN` edges on the same graph so agentless CVEs light up attack paths even when Inspector is disabled. Container image layers are reconstructed into a rootfs and scanned through the same pipeline; the block-plane for EBS volumes is present, with full userspace filesystem parsing as a gated follow-on. The scan is fail-closed on a partial or corrupt rootfs.

4.9 Kubernetes Posture (KSPM / KIEM)

Over a read-only Kubernetes API seam (`aws_kube.py`), OverWatch evaluates CIS-EKS posture and Kubernetes RBAC, and models cross-plane identity — IRSA / Pod-Identity `ServiceAccount`→`AWS role`→`admin/crown` relationships — folding Kubernetes objects (`KubePod`, `KubeServiceAccount`, `KubeAdminCapability`) into the same attack-path graph. Running Fargate tasks are fused in as well, with public-task-IP exposure surfaced.

4.10 Data Security Posture (DSPM) & AWS-Resident Secrets

DSPM (`aws_dspm.py` / `aws_deepplane.py`) classifies crown-jewel data across a dozen datastore kinds (`S3`, `RDS/Aurora`, `DocumentDB`, `Neptune`, `MemoryDB`, `FSx`, `Kinesis`, `Timestream`, `OpenSearch`, and more) from tags and Macie signal, normalizing heterogeneous sensitivity into data-type and tier labels at read time and computing a crown-jewel inventory, exposure, and classification-coverage gap. A crown Secret node (`Secrets Manager` + `SSM SecureStrings`) with a `CAN_READ_DATA` reader edge makes `internet`→`workload`→`role`→`secret` a first-class attack path. All of this is metadata-only — OverWatch never reads a secret value or a data object.

4.11 AI Security Posture (AI-SPM)

AI-SPM (`aws_aispm.py`) classifies the blast radius of an AI resource's execution role — whether it is privilege-escalation capable, reaches crown-jewel data, or lacks network isolation — and fuses that onto the graph, emitting AI posture findings and a flagship AI attack path where the AI execution role is a pivot to sensitive data.

4.12 The Runtime Ingest Planes (CDR / EDR / Malware)

Building a runtime sensor would break the charter, so OverWatch instead ingests the customer's existing runtime signals and correlates them onto the graph. A shared “CDR-lite” spine (`aws_cdr.py`) normalizes detections, folds them onto the stored graph as `THREAT_ON` annotations, and re-runs reachability so a detection on an `internet→crown/admin` path escalates into a ranked incident:

- **CDR.** GuardDuty, Security Hub ASFF, and CloudTrail-anomaly detections.
- **EDR / runtime sensor ingest.** The customer's existing CrowdStrike / Falco / GuardDuty-
Runtime / OCSF output; a sensor-inventory feed drives `runtime_monitored` coverage (and an honest “blind-spot” view) — `aws_edr.py`.
- **Malware.** GuardDuty Malware Protection / ClamAV / YARA output, with a cross-engine hit when malware lands in a DSPM crown store — `aws_malware.py`.

Crucially, these fold in as `THREAT_ON` annotations — out of the traversable edge set — so the correlator stays frozen and the ingested signal escalates existing paths rather than inventing new lateral-movement hops.

4.13 Cloud-Forensics Timeline

For an individual resource, OverWatch can reconstruct a who-did-what-when timeline from read-only CloudTrail management events, correlated with the graph, findings, and detections (`aws_forensics.py`). It is behind an injected seam and fails open to an explicit note rather than fabricating a clean timeline.

4.14 Grounded-RAG Copilot

The copilot (`aws_copilot.py`) answers questions using only the account's own scan corpus — a self-contained BM25 retrieval over findings, paths, and choke points. It is extractive by default and abstains rather than hallucinating; an optional LLM seam can be injected to phrase answers, but the retrieved context is always the scan's own data, so nothing leaves the boundary.

5. Data Model & Persistence

5.1 One Backend, Two Dialects

All persistence routes through a single Backend abstraction (`cnapp_backend.py`). Opening the store with a `sqlite://` URL runs the whole state plane on a local file; opening it with a `postgresql://` URL runs the identical schema on PostgreSQL via `psycopg3` for high availability. The SQL is byte-identical across dialects, and a missing driver or unreachable server fails loudly rather than silently falling back to a local file. Schema is versioned and migrated forward on open.

5.2 What Is Persisted

- **Onboarding registry.** Accounts, scan jobs, and connection health (`cnapp_registry.py`).
- **Scan results & posture.** Findings, the finding catalog, the full graph, attack paths, and the posture score per account.
- **Finding lifecycle.** Drift, waivers, and posture history over time (`aws_state.py`).
- **SBOM snapshots.** Content-addressed, diffable SBOM documents for supply-chain analysis.
- **Ingest tiers.** Owned external CVEs, detections/incidents, and runtime-sensor inventory.
- **Multi-tenancy.** Workspaces, members, platform admins, and the account↔workspace binding; a fail-open usage-metering ledger.

5.3 Display-Only Configuration (No Schema Churn)

Several capabilities are intentionally config-driven and hold no database schema at all: Projects (business-impact groupings), Controls (saved queries), Policies (policy-as-code), and the non-AWS registry connectors. Each is loaded from an environment variable (a JSON list or file path), validated fail-safe, and overlaid at read time. This keeps the frozen core and the schema stable while the surface grows.

6. Query & Governance Layer

6.1 WQL — the Security-Graph Query Language

WQL (`aws_wql.py`) gives the graph a queryable surface without a Gremlin/Neptune backend, which would break the self-hosted / air-gap charter. A query is a closed, typed JSON object — no free text, no regex, no eval — compiled to the frozen `aws_graph` traversal primitives. `parse()` is the security boundary: any unknown field, operator, predicate, or non-whitelisted property is rejected, and limit/max-hops are clamped, so a query can never escape the grammar or run unbounded. `evaluate()` returns deterministic, code-point-sorted node rows.

```
// "crown-jewel S3 buckets reachable from the internet" – the toxic-combination query
{ "kind": "S3Bucket",
  "where": { "op": "and", "of": [
```

```
{ "pred": "crown_jewel" },  
{ "pred": "reachable_from", "target": "internet" } ] } }
```

6.2 Controls & Policy-as-Code

A Control turns a saved WQL query into governance: a matching query overlays a display-only WARN finding into the catalog. Policy-as-code (`aws_policy.py`) is the in-chapter substitute for Rego/OPA (a Go binary subprocess would break zero-telemetry / air-gap): a closed, typed pure-Python DSL that combines a finding-catalog predicate (compliance-as-code — match by check id, section, severity, or compliance control) with a graph clause (WQL), combined any/all. A firing policy overlays a display-only WARN POLICY-xx finding. All of this is read at request time and never re-runs scoring.

6.3 SAMPLE == LIVE Parity

The console can run in a demo (SAMPLE) mode with no backend, which means several matchers exist twice — once in Python (the engine oracle) and once in TypeScript (the client mirror: `wql.ts`, `controls.ts`, `dspm.ts`, `policy.ts`). To guarantee they never diverge, a shared cross-language parity fixture is generated from the Python oracle and replayed by the TypeScript tests; CI fails on any drift. This discipline — sample matchers must mirror the Python byte-for-byte — is a recurring source of correctness in the platform.

7. Supply Chain & Container Registry

7.1 SBOM Ingest, Diff, License & VEX

OverWatch ingests SBOMs (CycloneDX / SPDX) and CI-pushed scanner output (SARIF), owns each CVE onto a graph node, enriches it from OverWatch's own OSV/EPSS/KEV bundle, and re-runs reachability so CVEs rank by attack-path exploitability rather than raw CVSS. SBOM snapshots are content-addressed and diffable; a license policy and standalone OpenVEX/CSAF-VEX suppression complete the supply-chain picture.

7.2 Agentless ECR Registry Scanning

Beyond CI-pushed SBOMs, OverWatch scans the registry itself. Tier A (always on, no new grant) widens the native ECR scan-finding sweep to the newest tagged images per repository. Tier B (opt-in, behind a two-key grant) pulls image layers, reconstructs the rootfs, and runs OverWatch's own SBOM→OSV pipeline — Inspector-independent and fail-closed on a partial rootfs. Native and own-SBOM CVEs converge on the same ECRImage node, split by a scan-source chip, and persist as diffable snapshots; a registry-only (undeployed) image carries `HAS_VULN` but never enters an attack path.

7.3 Non-AWS OCI Registry Connectors

The same agentless pull-and-scan extends to non-AWS OCI registries — GitHub GHCR, Docker Hub, Harbor, and Azure ACR — through one Docker Registry v2 Bearer-realm adapter

(aws_registry_oci.py) driven entirely by injected HTTP seams; a single adapter covers all four (including ACR's service principal, which authenticates as plain Basic). Connectors are declared in CNAPP_REGISTRIES with secret-ref-only credentials (aws_registry_connectors.py). Critically, the pull reuses the same allowlisted egress file as ECR — so the zero-telemetry allowlist does not grow — under a per-call host allowlist (config host plus the auth realm learned at runtime, never “any https”). A blob redirect is followed only to a non-SSRF-target HTTPS host (IMDS, cloud-metadata, loopback, and any private address are refused, with IP-literal encodings normalized), and the Authorization header is stripped on a cross-host hop. Pulled images run through the same registry-agnostic scan path as ECR, so the fail-closed-on-partial-rootfs contract cannot diverge; results are display-only.

Security note. The non-AWS registry slice was the single charter-sensitive edit in the platform (it loosens the ECR-only egress guard to a per-call allowlist). It was built as an isolated change and passed a dedicated read-only adversarial-verification pass; every confirmed finding — including an SSRF IP-encoding bypass and a credential-misroute — was fixed and regression-tested before merge.

8. Hosted Control Plane

The control plane (cnapp_service.PlatformService and the cnapp_* modules) wraps the engine with the operational machinery a multi-account, multi-tenant deployment needs.

- **Onboarding.** Mints a per-account ExternalId (stored only as a secret reference), builds the CloudFormation launch URL/CLI, and records the account as pending→active as the trust is validated (cnapp_onboarding.py, cnapp_validate.py).
- **Scheduling & workers.** Due-account scan scheduling and async scan-job execution that pre-validates credentials and re-checks trust at run time (cnapp_worker.py).
- **Multi-tenancy (MSSP).** Isolated, metered tenant workspaces with a workspace-scoped account binding and gate (cnapp_workspace.py).
- **RBAC.** Fail-closed, workspace-scoped roles (viewer / ingest / admin / superadmin) enforced at the API boundary; a cross-tenant account access resolves to 404, not 403 (cnapp_api.py).
- **Usage metering.** A fail-open, append-only, exactly-once ledger; billable = accounts under management (cnapp_metering.py).
- **Notification connectors.** Route findings to the operator's OWN Jira / Slack / PagerDuty / Splunk / webhook, with pure renderers, an idempotent delivery ledger, and secret-ref-only credentials (cnapp_connectors.py).

9. External Interfaces (HTTP API)

All routes delegate to PlatformService; admin routes stay on the private control plane. The table summarizes the surface by domain (see Appendix C for the full catalog).

Domain	Representative routes
Accounts & scans	POST /accounts, POST /accounts/{id}/validate, POST /scans, GET /scans/{job}
Findings & graph	GET /accounts/{id}/{summary,findings,paths,graph}, GET /org/{overview,findings}
Graph query	POST /accounts/{id}/graph/query (WQL), GET /controls, GET /policies
Ingest	POST /accounts/{id}/ingest, .../detections, .../edr/ingest, .../malware/ingest
Data & runtime	GET /accounts/{id}/data/inventory, .../edr/coverage, GET /org/incidents
Supply chain	GET /accounts/{id}/sbom/{subjects,snapshots,diff}, .../components, .../vex
Non-AWS registries	GET /registries, GET /registries/images, POST /registries/{id}/scan
Multi-tenancy	POST/GET /workspaces, /admin/platform-admins, /admin/usage

Fail-closed. Every route returns 403 until a real `current_principal` and a secret resolver are injected. Do not expose the hub before wiring authentication.

10. Frontend Architecture

The web console is a React single-page application built to static assets and served by the hub (CNAPP_STATIC_DIR). Screens are source-agnostic: the same fetch shape backs both LIVE mode (the FastAPI hub) and SAMPLE mode (bundled fixtures), toggled at build time. In SAMPLE mode the client evaluates the same matchers the engine does — the WQL, Controls, DSPM, and Policy logic are mirrored byte-for-byte in TypeScript and guarded by the cross-language parity fixture described in §6.3. Primary screens include Overview, Attack Paths (the differentiator, led first), Findings, Vulnerabilities, Runtime, Data Security, Supply Chain, Registries, Query, Identity, Compliance, Remediation, and Reports.

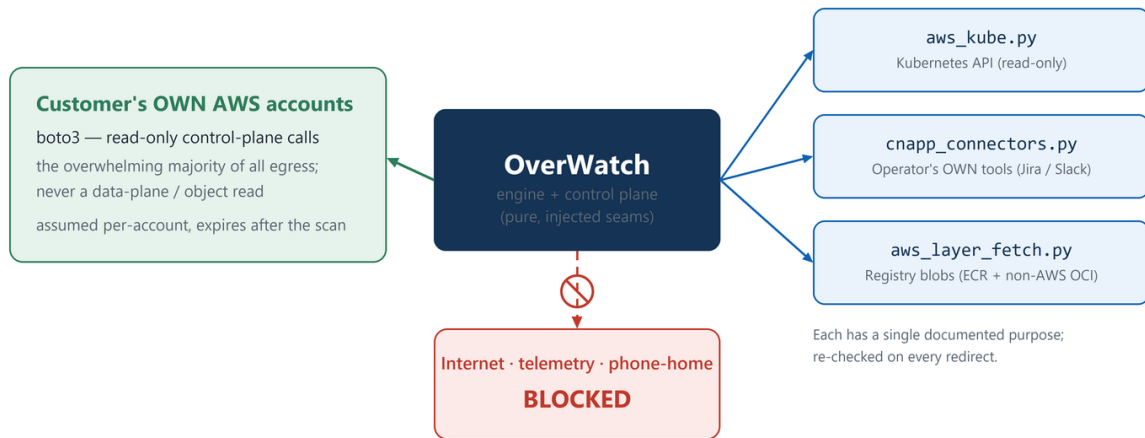
11. Security Architecture

11.1 Egress Containment & Zero-Telemetry

The single most important security property is that OverWatch's own egress is tiny and auditable. Only the three allowlisted files of §2.2 may open a socket, each enforced by the recursive-AST tripwire test. Every other outbound call is a boto3 request to the customer's own account. The complete inventory is in NETWORK.md.

Figure 5 — Egress Containment (zero-telemetry)

Only three files may open a socket; enforced by a recursive-AST tripwire test



Guard: `tests/test_zero_telemetry.py` — a recursive-AST scan of every shipped module fails CI if any other file holds a network primitive.

Figure 5 — Egress containment: the bulk of egress is read-only boto3 to the customer's own accounts; only three allowlisted files may open any other socket; internet / telemetry is blocked.

11.2 Secret Handling

Operator credentials (connector tokens, registry pull secrets, the account ExternalId) are always stored as `secretsmanager://` or `ssm://` references and resolved to plaintext only transiently at the point of use. Plaintext is never persisted, never logged (error strings are scrubbed of any secret substring), and never returned over the API (masked to a `secret_configured` boolean).

11.3 SSRF & Redirect Guards

The registry egress file enforces HTTPS-only, a per-call host allowlist re-checked on every redirect, a byte cap, a short-read fail-close, and an SSRF guard that normalizes IP-literal encodings (IPv4-mapped IPv6, integer, hex, 0.0.0.0, ::) via the `ipaddress` module — so a dotted-decimal-only blocklist cannot be evaded — and strips the Authorization header on any cross-host hop.

11.4 IAM Trust Model

The hub runs under an instance profile that lets it assume a read-only role (the SecurityAudit managed policy plus a few explicit read grants) in each spoke account. Cross-account access is gated by an ExternalId. The scanner never holds long-lived spoke credentials; it assumes, reads, and lets the session expire.

11.5 Fail-Closed Authentication

The API ships fail-closed: without an injected `current_principal` (from the customer's IdP/JWT) and a secret resolver, every route is denied. A production deployment terminates TLS and

enforces identity at an internal load balancer in front of the hub, and scopes the instance security group to that load balancer rather than to a broad CIDR.

12. Deployment & Operations

12.1 Topology

OverWatch is delivered as a hardened, non-root container that runs the control plane and engine together and serves the API/console on port 8080. The reference deployment places one instance on a private subnet in the security VPC with a KMS-encrypted state volume, IMDSv2-required metadata, and an egress-restricted security group.

12.2 The Shipped Deploy Path

Step	Artifact	Purpose
1	deploy/cnapp-hub-role.yaml	Create CnappHubRole + CnappHubInstanceProfile (the hub identity)
2	deploy/marketplace/hub-deploy.yaml	Launch the hub EC2 instance (private subnet, KMS volume, SG, container)
3	deploy/cnapp-scanner-role.yaml	Deploy the read-only role into each spoke account (or via StackSet)

12.3 Configuration (Environment Variables)

Variable	Purpose
CNAPP_DB_URL	State backend: sqlite:///data/overwatch.db (default) or postgresql://...
CNAPP_HUB_ROLE_ARN	The hub role the instance assumes-from into spokes
CNAPP_STATIC_DIR	Path to the prebuilt SPA served by the hub
CNAPP_VULN_DB	Path to the (optionally signed) BYO OSV/EPSS/KEV feed
CNAPP_PROJECTS / _CONTROLS / _POLICIES / _REGISTRIES	Config-driven overlays (JSON list or file path; fail-safe)
OVERWATCH_AIRGAP	Documents the sealed posture (all optional seams already default off)

12.4 Air-Gapped Install

For a sealed environment, a connected build host produces exactly one artifact (scripts/build_offline_bundle.sh → a tarball containing the Docker image, the pinned wheelhouse, the prebuilt SPA, and the deploy artifacts). On the sealed side the image is docker-

loaded, VPC interface endpoints are provisioned for the AWS services the engine calls (sts, ec2, iam, s3-gateway, logs, secretsmanager, ssm, cloudtrail, guardduty, securityhub, access-analyzer, eks, config, kms), and an optional signed vulnerability feed is mounted and verified with a public key. The full procedure is docs/AIRGAP_RUNBOOK.md.

12.5 Amazon Linux Notes

- **Install a container runtime.** The shipped UserData assumes docker is present; on Amazon Linux 2023 add it first (dnf install -y docker && systemctl enable --now docker), or use an ECS-optimized / Bottlerocket AML.
- **Run the container, not the host Python.** Amazon Linux ships an older system Python; the image carries its own Python 3.12, so there is no need to install a newer interpreter on the host.

12.6 Shift-Left: CI/CD Gate & IDE

A shell-only CI/CD image-scan GitHub Action and an egress-free IaC policy-gate Action let the same engine run left of production; the offline IaC scanner emits SARIF with real file:line and gates a pull request on a severity floor or a policy. An IDE reference stub surfaces the same diagnostics inline.

13. Implementation Guide

13.1 Repository Layout

The engine modules (aws_*.py) are pure and boto3-injected; the control-plane modules (cnapp_*.py) wrap them with persistence and the API; the frontend/ tree holds the React SPA and its TypeScript mirrors; deploy/ holds the CloudFormation and container artifacts; tests/ mirrors the source. Appendix A lists the modules.

13.2 The Core Pattern: Pure Module + Injected Seam

Every new capability should be a pure module whose I/O (AWS session, HTTP, secrets, clock) is injected, so it is unit-testable offline. Follow the existing engines: compute over a data structure (usually the graph or the finding catalog), return values, and let the caller do the I/O.

13.3 Adding a Posture Check

- Emit a finding with a stable check id, section, severity, and a full risk / impact / step-by-step remediation write-up (the finding-detail contract).
- If it changes reachability or privilege, add or prune the corresponding graph edge so it participates in attack paths; otherwise keep it a flat finding.
- Add a mock-boto3 unit test; if the check has a client-side mirror, extend the parity fixture.

13.4 Adding an Ingest Normalizer or Overlay

- Normalize the external format to the shared detection/finding shape and fold it onto the graph as an annotation (e.g., THREAT_ON) — never as a traversable edge.
- Keep it display-only: overlays are appended at read time and must not touch the posture score or the frozen correlator.
- For a config-driven overlay, load it fail-safe from an environment variable and mask any secret in API views.

13.5 The SAMPLE == LIVE Rule

If a matcher runs client-side in demo mode, it must mirror the Python oracle byte-for-byte and be covered by the cross-language parity fixture. Historically, most correctness regressions in the platform trace to a client mirror drifting from the Python — the fixture is the guard, and adding a case to it is part of the change, not an afterthought.

13.6 Coding Conventions

- Python 3.10+; type hints on public functions; structured logging, never bare print.
- One responsibility per module; pure core, injected I/O; no module-level side effects (the API is a factory so import has no disk effect).
- Never widen the egress allowlist; never add a heavy or non-pure dependency to the air-gap wheelhouse.

14. Testing & Quality Assurance

14.1 The Suite

The backend suite is 2169 tests that run with mock boto3 and no AWS credentials; the frontend has 181 vitest tests plus a type-check and build. A regression test backs every defect the platform's adversarial-verification passes have found.

14.2 The Guards

Guard	What it enforces
tests/test_zero_telemetry.py	Recursive-AST tripwire: only the 3 allowlisted files hold a network primitive
Correlator checksum test	aws_correlate is byte-frozen (traversable-edge set + crown logic)
Cross-language parity fixture	Python oracle == TypeScript client mirror (WQL/Controls/DSPM/Policy)
Fail-closed / partial-data tests	Partial rootfs, truncated layer, unsigned feed all fail closed

14.3 The Rigor Loop

Substantive changes follow a repeatable loop: scope the change, build it in small green increments, run a read-only adversarial-verification pass that tries to refute each candidate finding, fix every confirmed issue with a regression test, update docs, and only then present for commit (feature branch → no-fast-forward merge → push). This is how the non-AWS registry slice shipped with its SSRF and credential-handling issues found and fixed before merge.

15. Roadmap & Extensibility

The eight-phase vulnerability/misconfiguration detection roadmap is complete, and the Wiz-parity “coverage-close” program (surface-over-engines, WQL/Controls, EDR ingest, data/malware, policy-as-code + CI gate, and non-AWS registries) has shipped. Natural extension points that respect the charter include: completing the EBS userspace filesystem parse for full VM side-scan; additional read-only Kubernetes and multi-cloud read surfaces; more ingest normalizers; and richer report/export targets. Anything requiring an agent, active exploitation, a data-plane read, or new telemetry egress is intentionally out of scope.

Appendix A — Module Inventory

Module	Responsibility
aws_live_scanner.py	Live audit scanner entrypoint (296 checks / 44 sections; orchestrates the engine)
aws_graph.py	The security graph (typed nodes + edges)
aws_exposure.py	Internet exposure + L7 four-gate reachability oracle
aws_deepplane.py	Deep-plane ingestion (Inspector/Macie/GuardDuty/Access Analyzer) + DSPM read surface
aws_correlate.py	Attack-path correlation & choke points (byte-frozen)
aws_effperm.py	Effective-permissions solver (identity $\cap$ boundary $\cap$ SCP)
aws_leastpriv.py	CIEM right-sizing + least-privilege policy generation
aws_sidescan*.py	Agentless CWPP: inventory + OSV match + secrets + image layer overlay
aws_kube.py	KSPM/KIEM over a read-only Kubernetes API seam (egress-allowlisted)
aws_dspm.py / aws_secrets.py	Data security posture + AWS-resident secrets (metadata only)
aws_aism.py	AI security posture (execution-role blast radius)
aws_cdr.py / aws_edr.py / aws_malware.py	Runtime / EDR / malware ingest (THREAT_ON, out of E_PATH)
aws_forensics.py	CloudTrail-based cloud-forensics timeline
aws_copilot.py	Grounded-RAG copilot over the scan's own corpus
aws_ingest.py / aws_sbom_diff.py / aws_license.py / aws_vex.py	External-vuln & supply-chain ingest, diff, license, VEX
aws_wql.py / aws_controls.py / aws_policy.py	Graph query language + Controls + policy-as-code
aws_registry_sbom.py / aws_sidescan_image.py	ECR registry side-scan; shared registry-agnostic scan path
aws_registry_oci.py / aws_registry_connectors.py	Non-AWS OCI registry pull adapter + connector config
aws_layer_fetch.py	The sole registry layer-blob egress seam (ECR + OCI), allowlisted
aws_state.py / cnapp_backend.py	Finding lifecycle/drift/waivers + dual-dialect backend
cnapp_service.py / cnapp_api.py	PlatformService facade + FastAPI routes (fail-closed RBAC)

cnapp_onboarding / _validate / _registry / _worker	Onboarding, trust validation, account registry, scan workers
cnapp_workspace.py / cnapp_metering.py	Multi-tenancy (workspaces) + usage metering
cnapp_connectors.py	Outbound notification connectors (allowlisted egress)

Appendix B — Graph Model Reference

Traversable attack edges (E_PATH)

EXPOSED_TO • ATTACHED_TO • HAS_INSTANCE_PROFILE • HAS_ROLE • CAN_ASSUME • CAN_PRIVESC_TO • CAN_READ_DATA • TARGETS

Annotations (enrich a node; never walked as a hop)

HAS_VULN • THREAT_ON • RUNS_IMAGE • HAS_TECH (and other non-traversable relationships)

Node kinds are listed by category in §4.2. The distinction between the traversable set and annotations is the mechanism that keeps ingested signal (vulnerabilities, threats, technologies) additive without destabilizing attack-path ranking.

Appendix C — Selected API Routes

Method & path	Role
POST /accounts	admin
POST /accounts/{id}/validate	admin
POST /scans	admin
GET /scans/{job_id}	viewer
GET /accounts/{id}/{summary,findings,paths,graph}	viewer
POST /accounts/{id}/graph/query	viewer
GET /controls, GET /policies	viewer
POST /accounts/{id}/ingest	ingest
POST /accounts/{id}/edr/ingest	ingest
GET /registries, GET /registries/images	viewer
POST /registries/{id}/scan	admin
GET /org/{overview,findings,incidents}	viewer
/workspaces, /admin/*	admin / superadmin

Appendix D — Glossary

Term	Definition
Crown jewel	A high-value target (sensitive data store, admin capability) an attack path aims for
Attack path	An entry→target chain over traversable edges; the unit of risk ranking
Choke point	A node whose remediation severs many paths to many crown jewels
E_PATH	The frozen set of edge kinds that are walked when computing attack paths
Deep plane	AWS-native detectors (Inspector/Macie/GuardDuty/Access Analyzer) ingested as signal
Display-only overlay	A read-time WARN finding that never affects posture or the graph
Hub / spoke	The self-hosted OverWatch instance / an audited AWS account
CWPP / DSPM / CIEM / AI-SPM	Workload / data / entitlement / AI security posture pillars